



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE ENSINO SUPERIOR DO SERIDÓ
BACHARELADO EM SISTEMAS DE INFORMAÇÃO.

JUAN VYCTOR SILVA GARCIA DE OLIVEIRA

TRABALHO AVALIATIVO DE ESTRUTURA DE DADOS

CAICÓ - RN
2026

JUAN VYCTOR SILVA GARCIA DE OLIVEIRA

TRABALHO AVALIATIVO DE ESTRUTURA DE DADOS:

Atividade do curso de graduação em Bacharelado em Sistemas de Informação, como parte dos requisitos para obtenção de nota na matéria de Estrutura de Dados.

Orientador(a): Dr. Arthur Emanuel Cassio da Silva e Souza.

Co-orientador(a): .

CAICÓ - RN

2026



Esta obra está licenciada com uma licença Creative Commons Atribuição 4.0 Internacional. Permite que outros distribuam, remixem, adaptem e desenvolvam seu trabalho, mesmo comercialmente, desde que creditem a você pela criação original. Link dessa licença: <<https://creativecommons.org/licenses/by/4.0/legalcode>>

RESUMO

Este trabalho apresenta uma análise experimental de algoritmos recursivos e iterativos aplicados aos problemas de Fibonacci, Fatorial e algoritmos de ordenação. O objetivo foi comparar os tempos de execução e relacionar os resultados obtidos com as estimativas de complexidade assintótica Big-O. Para isso, foram realizados experimentos práticos utilizando diferentes entradas, permitindo a geração de gráficos comparativos entre as abordagens analisadas. Os resultados evidenciam diferenças de desempenho entre implementações recursivas e iterativas, destacando o impacto da complexidade computacional na eficiência dos algoritmos.

Palavras-chave: Recursão; Iteração; Complexidade de Algoritmos; Big-O; Algoritmos de Ordenação

ABSTRACT

This work presents an experimental analysis of recursive and iterative algorithms applied to Fibonacci, Factorial, and sorting problems. The objective was to compare execution times and relate the obtained results to Big-O asymptotic complexity estimations. Practical experiments were conducted using different input sizes, allowing the creation of comparative graphs between the analyzed approaches. The results highlight performance differences between recursive and iterative implementations, emphasizing the impact of computational complexity on algorithm efficiency.

Keywords: Recursion; Iteration; Algorithm Complexity; Big-O; Sorting Algorithms

SUMÁRIO

1	INTRODUÇÃO	5
2	CONFIGURAÇÕES DE AMBIENTE	6
3	EXPLICAÇÃO DO EXPERIMENTO	6
3.1	Processo	6
3.2	Algoritmos Escolhidos	6
4	EXECUÇÃO	7
4.1	Fibonacci	7
4.2	Fatorial	8
4.3	Ordenação N^2	9
4.4	Ordenação $n \log n$	10

1 INTRODUÇÃO

O estudo de algoritmos recursivos e iterativos é fundamental na área de Estrutura de Dados e Análise de Algoritmos, pois permite compreender como diferentes abordagens influenciam diretamente o desempenho computacional. Problemas clássicos, como os algoritmos de Fibonacci, Fatorial e métodos de ordenação, são frequentemente utilizados para demonstrar conceitos relacionados à eficiência, complexidade e otimização de código.

Neste trabalho, foi realizada uma análise experimental comparando implementações recursivas e iterativas desses algoritmos. A metodologia adotada seguiu as orientações propostas no documento da atividade, sendo baseada na medição empírica dos tempos de execução. Para isso, foram utilizados os códigos disponibilizados juntamente à tarefa, executando experimentos com diferentes tamanhos de entrada a fim de observar o comportamento dos algoritmos em cenários variados.

Durante os experimentos, foram coletados dados de desempenho para cada algoritmo avaliado, possibilitando a construção de gráficos individuais e comparativos entre as versões recursivas e iterativas. Além disso, foi realizada a estimativa da complexidade assintótica Big-O de cada implementação, relacionando os resultados teóricos com os dados obtidos experimentalmente.

Por meio dessa análise, busca-se compreender as diferenças de eficiência entre as abordagens estudadas, evidenciando como fatores como repetição de chamadas, uso de memória e crescimento do tempo de execução impactam o desempenho geral dos algoritmos.

2 CONFIGURAÇÕES DE AMBIENTE

A máquina utilizada para os experimentos foi um MacBook Pro 2017, com um processador Intel Dual Core i5 2,3GHz, placa gráfica Intel Iris Plus Graphics 640 1536 MB, uma memória RAM 8 GB 2133 MHz LPDDR3, e um sistema operacional macOS Ventura versão 13.7.8.

Em relação ao Python, a versão instalada na máquina é a 3.12.2, atingindo o requisito requerido pela tarefa, e a biblioteca matplotlib está em sua versão 3.10.9.

3 EXPLICAÇÃO DO EXPERIMENTO

3.1 Processo

O experimento foi feito utilizando os códigos proporcionados pelo Professor, seguindo suas instruções e indicações, a fim de que tudo fosse executado de maneira correta. Ao executar o arquivo ExecExperimento, é possível escolher quais dos algoritmos seria utilizado, juntamente de quantas entradas o mesmo receberia. Dessa forma, é possível adquirir dados diversos sobre eficiência, tempo gasto e complexidade.

3.2 Algoritmos Escolhidos

Juntamente dos algoritmos de cálculo de números fatoriais e sequência de Fibonacci, ambos providenciados pelo Professor, era necessário que o aluno buscasse outros dois algoritmos diferentes, um de ordem de complexidade $O(N^2)$, e outro de complexidade $O(n \log n)$, para que pudessem ser comparados e discutidos. Nesse trabalho, serão abordados as ordenações CombSort, de ordem $O(N^2)$, e o HeapSort, $O(n \log n)$.

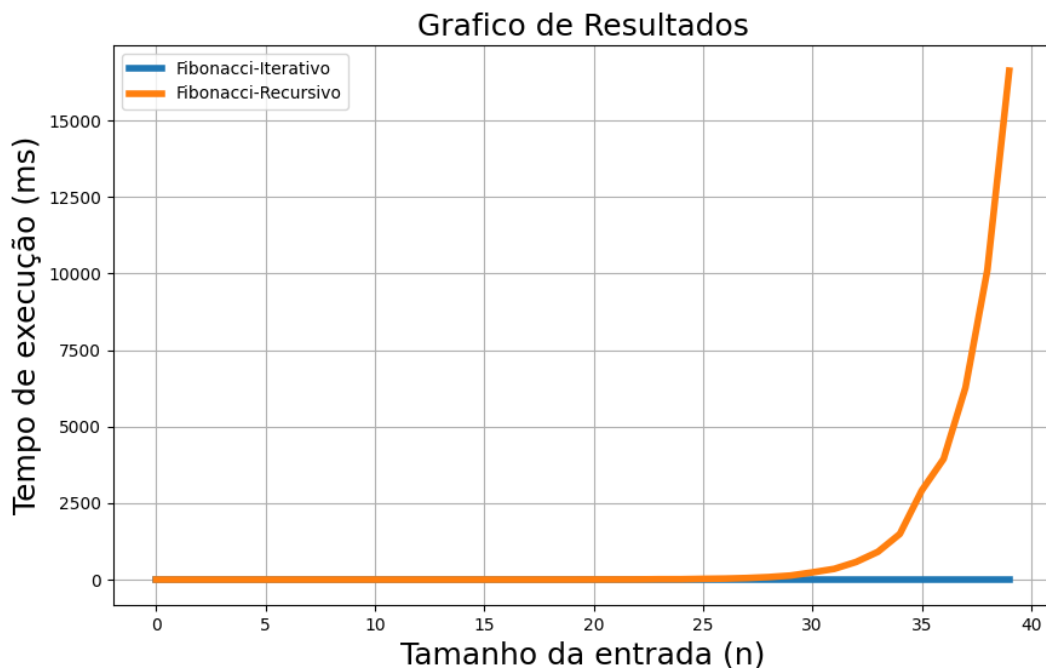
O CombSort é uma ordenação inspirada no BubbleSort, onde a principal diferença é o método de comparação. Onde o BubbleSort compara os valores adjacentes um à um, sendo necessárias diversas iterações para corrigir uma lista, o CombSort compara valores inicialmente bem distantes, para economizar o máximo de tempo possível, e diminuindo essa distância pouco a pouco, para que no fim faça como o original, comparando os mais próximos.

Já o HeapSort é baseado num algoritmo Heap Binário, inicialmente organizando o vetor como uma árvore, removendo a raiz, que pode ser o maior ou menor valor, ordenando a ponta, fazendo uso do Heap Binário para se reorganizar, e repetindo. Dessa forma, ao fim do processo, a lista fica completamente ordenada de forma rápida, já que o Heap tem complexidade $\log n$, e a ordenação é simplesmente N .

4 EXECUÇÃO

4.1 Fibonacci

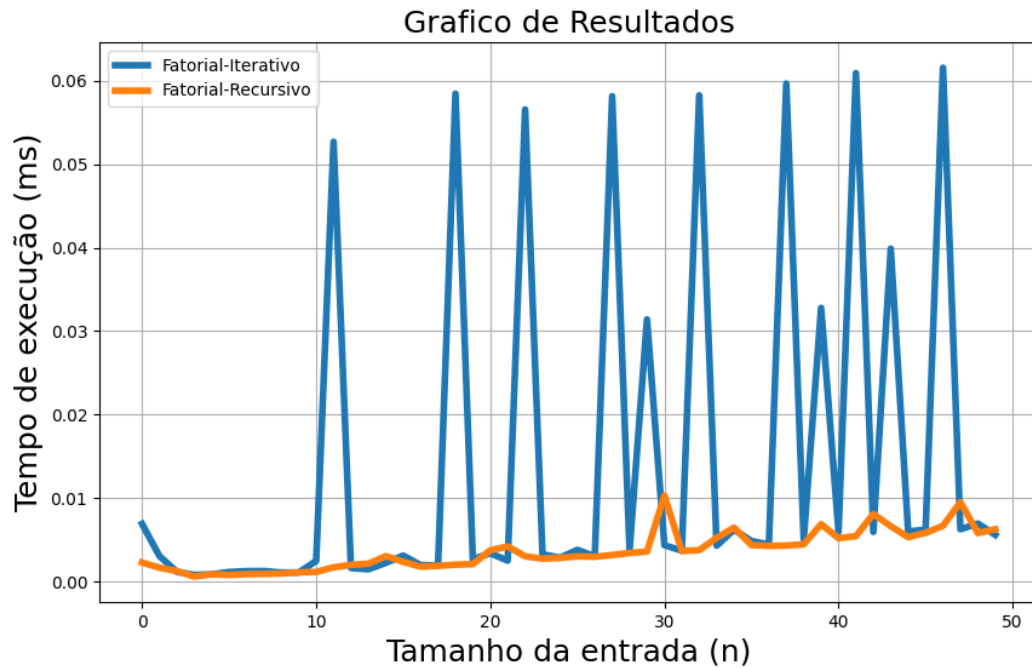
Inicialmente foi feito o experimento com a sequência de Fibonacci, um algoritmo clássico. Em azul está uma versão iterativa do código, e em laranja, recursiva. A coisa mais notável no gráfico é o comportamento do modo recursivo, crescendo rapidamente a partir de um certo ponto, onde sua complexidade N^2 faz as instruções se tornarem numerosas demais.



Até aproximada 30 entradas, a execução foi praticamente instantânea, a ponto da linha ser praticamente reta a olho nu. Após isso, a versão iterativa se torna muito mais rápida, devido à quantidade de processos sendo chamados em pilha pela versão recursiva. O tempo levado para chegar no caso base do Fibonacci e para devolver essas respostas uma a uma até o caso inicial demora muito mais que uma simples iteração direta da operação.

4.2 Fatorial

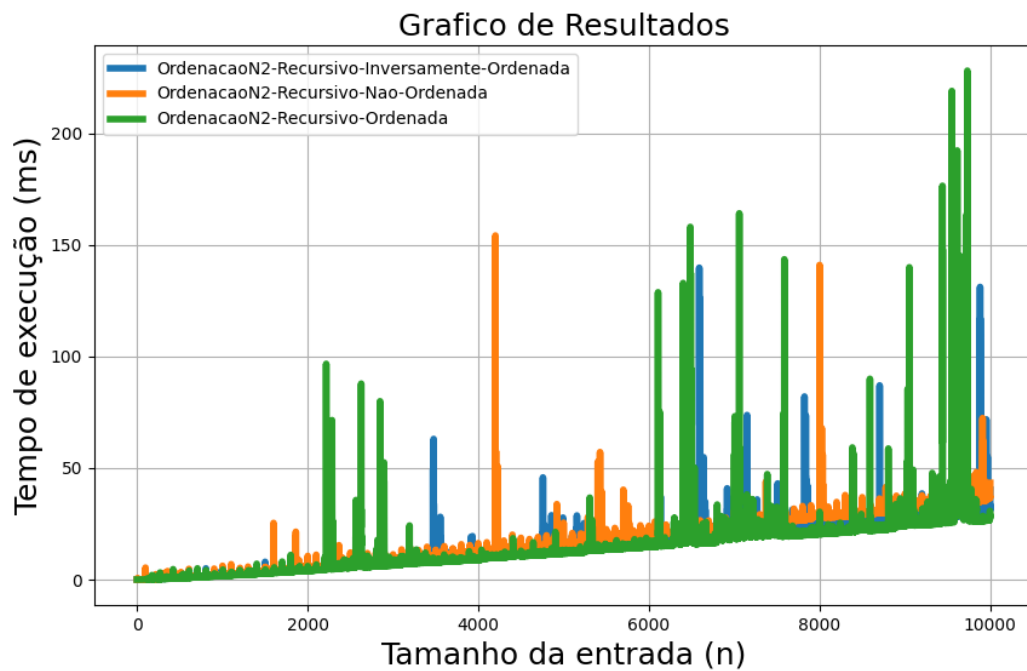
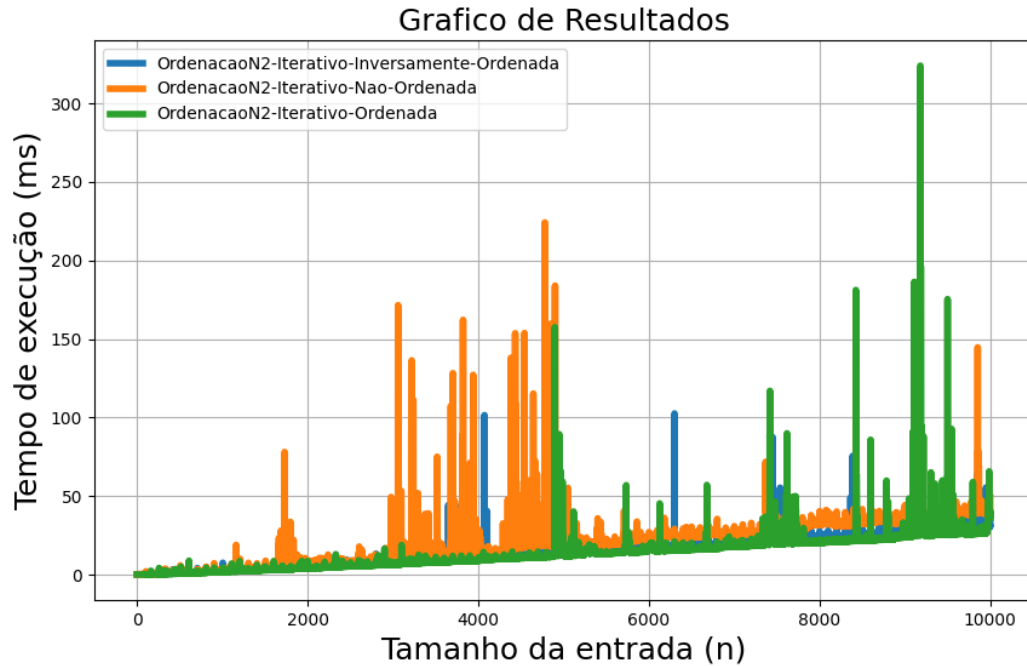
Em seguida, o foco foi o algoritmo de soma Fatorial. Com sua complexidade bem menor, apenas N no seu pior caso, é bem mais linear. Ambas as versões são bem próximas uma da outra, com as maiores diferenças sendo os gargalos da própria máquina ao fazer algum outro processo e dividindo sua memória e atrasando a execução principal.



É interessante ver a diferença entre ordens de complexidade. No Fibonacci, de função quadrática, apenas 40 entradas já fez o programa levar mais de 15 segundos por iteração, enquanto na Fatorial uma entrada de tamanho 50, não muito maior, mas considerável, não levou mais de uma pequena fração de segundo.

4.3 Ordenação N^2

Para o CombSort anteriormente descrito, todos os seus casos, melhor, médio e pior, são de ordem N^2 . Deste modo, as linhas do gráfico se mantêm bem próximas umas das outras, com apenas um pequeno desvio visível mesmo em 10000 entradas.



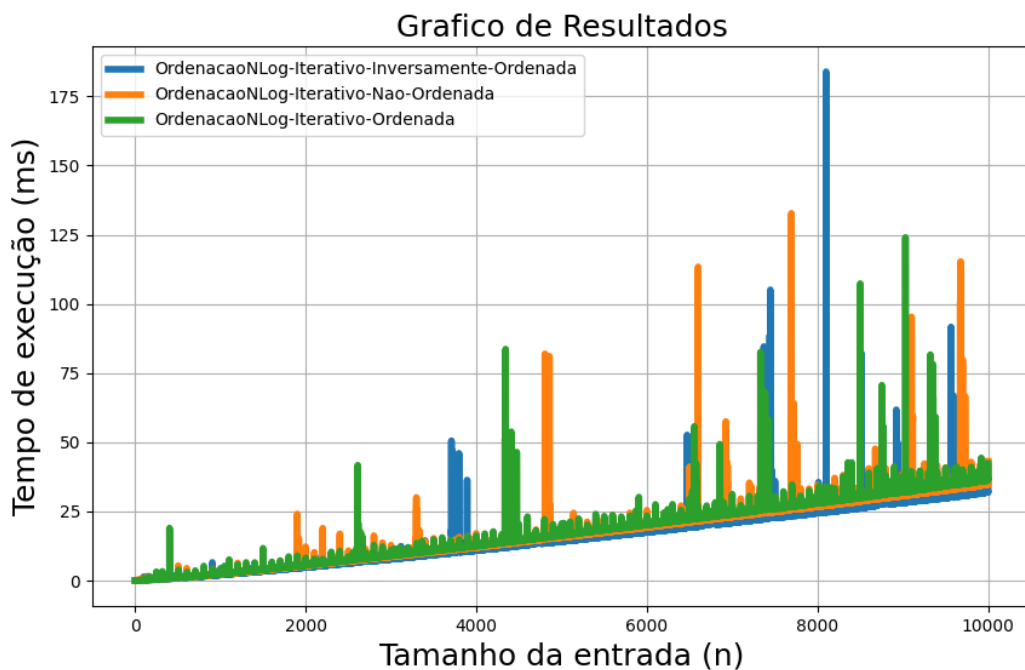
Analisando os gráficos e desconsiderando gargalos, se pode notar que a linha verde, com a lista ordenada, é o melhor caso, ficando um pouco mais abaixo das outras duas, devido ao fato das comparações de valores sempre retornarem "falso" para a verificação, e assim pulando alguns passos e economizando tempo.

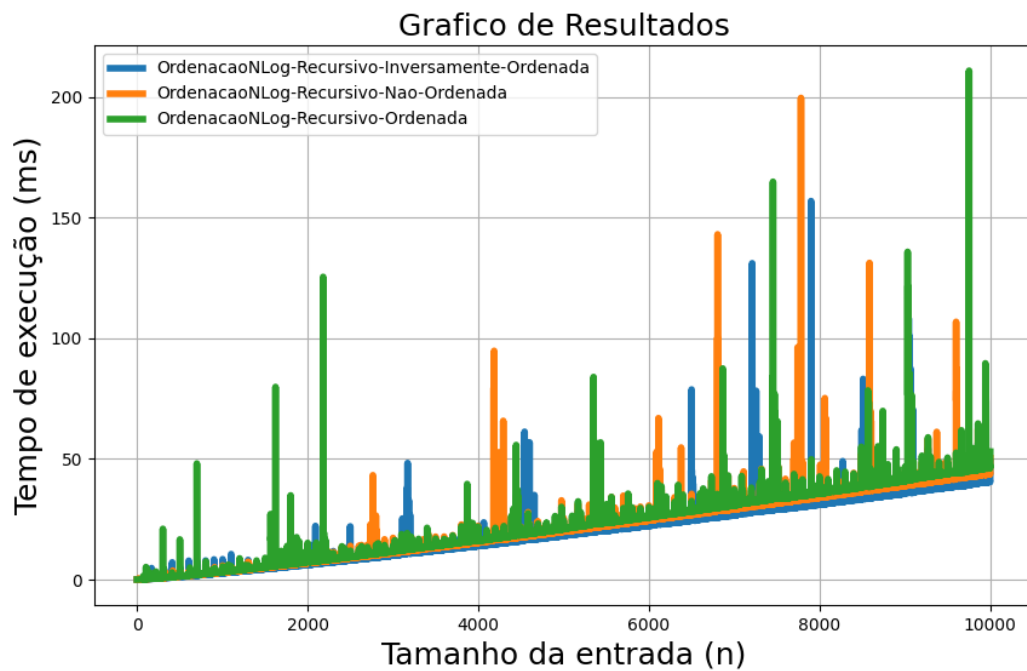
Os casos médio e pior são bem mais próximos, já que necessitam dessa checagem de valores, mas devido ao salto do CombSort, por vezes, uma lista aleatória pode ocasionar em etapas adicionais, por pular um valor menor na hora de alterar a ordem.

Por fim, é possível notar que a versão iterativa é levemente mais rápida que a recursiva, demonstrando a eficiência do código em ambos os casos.

4.4 Ordenação $n \log n$

No HeapSort, similar ao CombSort, seus casos também são de mesma ordem, independente dos valores, todos de ordem $n \log n$. Com isso, as 10000 entradas também ficam bem próximas, o que dificulta um pouco a visualização dos resultados.





Contrário ao anterior, no HeapSort o melhor caso é quando a lista está inversamente ordenada, já que a árvore já está separada, como ao fim do processo de heaping. Ao que mais e mais valores vão se tornando ordenados, a quantidade de processos e tempo vai aumentando.

Finalmente, a versão recursiva também é ligeiramente mais lenta que a iterativa, com pouca diferença, mas se tornando mais perceptível com o aumento de entradas.